Docker Commands:

Docker Run commands:

what does the docker run command like "docker run nginx" does, its definition, use, response understanding, when to use most often, alternate commands for it, etc

1️⃣ What `docker run nginx` does

The command:

docker run nginx

tells Docker to **create and start a container from the Nginx image**.

- `docker` → Docker CLI
- `run` → Creates a new container and starts it
- `nginx` → The image used to build the container

So internally Docker does these steps:

1. Check if the **Docker** daemon has the **Nginx** image locally
2. If not → it **pulls the image from** Docker Hub
3. Creates a **new container**
4. Starts the container
5. Runs the **default command defined in the image**

For Nginx, the default command is basically:

nginx -g "daemon off;"

which **starts the web server inside the container**.

2️⃣ Simple Definition (Interview-Ready)

`docker run` is a Docker command used to **create and start a new container from a specified image**.

It combines multiple operations into a single command:

- Pull image (if not present)
- Create container
- Start container
- Run the default process

3️⃣ Output / Response Understanding

If you run:

docker run nginx

You will usually see logs like:

/docker-entrypoint.sh: Configuration complete; ready for start up

This means:

Output Meaning	Explanation
Entry point script	Container initialization
Configuration complete	Nginx config loaded

Ready for startup	Web server running

But you **cannot access the website yet** because **no port is exposed**.

To access it:

docker run -p 8080:80 nginx

Now open:

http://localhost:8080

You will see the **Nginx welcome page**.

---

4 Real Structure of `docker run`

General syntax:

docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Example:

docker run -d -p 8080:80 --name my-nginx nginx

Meaning:

Option	Meaning
`-d`	Detached mode (run in background)
`-p`	Port mapping
`--name`	Container name
`nginx`	Image

---

5 When `docker run` is Most Often Used

Developers use it when they want to:

1 Quickly start a container

Example:

docker run redis

Run **Redis** server instantly.

---

2 Test applications

Example:

docker run node

Run **Node.js** environment quickly.

---

3 Run services

Example:

docker run -d -p 3306:3306 mysql

Run **MySQL** database.

---

4 Temporary containers

Example:

docker run ubuntu

Start **Ubuntu** shell container.

6️⃣ Important Options Used with `docker run`

Run in background

docker run -d nginx

Expose port

docker run -p 8080:80 nginx

Name container

docker run --name webserver nginx

Interactive terminal

docker run -it ubuntu bash

Auto delete container

docker run --rm ubuntu

Container deletes after stopping.

7️⃣ Alternate Commands (Related to `docker run`)

`docker run` actually combines several commands.

Equivalent steps:

Step 1 — Pull image

docker pull nginx

Step 2 — Create container

docker create nginx

Step 3 — Start container

docker start <container_id>

So internally:

docker run = docker pull + docker create + docker start

---

8 Common Commands Used After `docker run`

Check running containers:

docker ps

Stop container:

docker stop <container>

Remove container:

docker rm <container>

View logs:

docker logs <container>

---

9 Simple Workflow Example

Run Nginx container:

docker run -d -p 8080:80 --name nginx-server nginx

Check container:

docker ps

Output:

```
CONTAINER ID   IMAGE   PORTS
abc123         nginx   0.0.0.0:8080->80/tcp
```

Access:

http://localhost:8080

🔟 One-Line DevOps Interview Answer

> **`docker run` is used to create and start a container from a Docker image. If the image is not available locally, Docker pulls it from Docker Hub, creates a container, and runs the default process defined in the image.**

docker ps

1️⃣ What `docker ps` Command Does

The command:

docker ps

is used to **list all currently running containers** in **Docker**.

It shows important information about the containers that are **active and running on the system**.

2️⃣ Simple Definition (Interview-Ready)

`docker ps` is a command used to display a list of all running containers along with their details such as container ID, image, status, ports, and container name.

3️⃣ Output / Response Understanding

Example output:

```
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
a1b2c3d4e5     nginx     "/docker-entrypoint.…"   2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp     nginx-server
```

Explanation of each column:

Column	Meaning
CONTAINER ID	Unique ID of the container
IMAGE	Image used to create container (e.g., **Nginx**)
COMMAND	Default command executed in the container
CREATED	When the container was created
STATUS	Current state (running, exited, paused)
PORTS	Port mapping between host and container
NAMES	Container name

4️⃣ When to Use `docker ps`

Developers use this command when they want to:

1️⃣ Check if a container is running

Example:

```
docker ps
```

Use case: Confirm if **Nginx**, **Redis**, or **MySQL** containers are active.

2️⃣ Find container ID or name

Many Docker commands require **container ID or container name**, so `docker ps` helps retrieve them.

3️⃣ Debug applications

If an application is not working, DevOps engineers check running containers.

4️⃣ Verify port mappings

Example:

PORTS
0.0.0.0:8080->80/tcp

This means:

Host Port 8080 → Container Port 80

5️⃣ Why `docker ps` is Important

It helps to:

- Monitor running containers
- Identify container IDs
- Check container uptime
- Verify ports and services
- Debug container issues

In **DevOps workflows**, it is usually the **first command run after starting containers**.

6️⃣ Useful Variations of `docker ps`

Show all containers (running + stopped)

docker ps -a

Show only container IDs

docker ps -q

Show latest container

docker ps -l

Filter containers

docker ps --filter "status=running"

---

7 Alternative / Related Commands

While `docker ps` is the main command, some related commands help inspect containers.

List all containers

docker container ls

This is the **modern equivalent** of `docker ps`.

List all containers including stopped

docker container ls -a

Inspect container details

docker inspect <container_id>

Check container logs

docker logs <container_id>

---

8 Commands Commonly Used After `docker ps`

After identifying a container from `docker ps`, developers usually run:

Stop a container

docker stop <container_id>

Start a stopped container

docker start <container_id>

Restart container

docker restart <container_id>

Check logs

docker logs <container_id>

Enter container terminal

docker exec -it <container_id> bash

Remove container

docker rm <container_id>

---

9 Typical DevOps Workflow

Example workflow with **Docker**:

Start container:

docker run -d -p 8080:80 nginx

Check running containers:

docker ps

Enter container:

docker exec -it <container_id> bash

Stop container:

docker stop <container_id>

🔟 One-Line DevOps Interview Answer

> **`docker ps` is used to list all running Docker containers and display their details such as container ID, image, status, ports, and names. It is mainly used for monitoring and managing active containers.**

Assume you are expert in Docker and its key features, and a good guide to beginner then For the docker command "docker rm NAME", what does it and its sub commands or its alternative commands do, its use cases, when to use and its following commands, with best examples with explanation of commands for each to make beginner gets understand much easier, and other important things to know to crack interviews.

1. What is `docker rm`?

Command

docker rm CONTAINER_NAME_OR_ID

Purpose

`docker rm` **removes (deletes) a stopped container** from the Docker host.

Think of it like **deleting a file after you finished using it**.

- A **container must be stopped** before it can be removed.
- The **image remains intact**.

Example

docker rm my_container

What happens

1. Docker finds container **my_container**
2. If container is **stopped**
3. It **deletes the container metadata and filesystem**

Result:

my_container

---

2. Why `docker rm` is important (Real DevOps Use Case)

Containers are often created for:

- Testing
- Temporary environments
- CI/CD pipelines
- Automation tests (important for SDET)

After work is done, containers should be removed to avoid:

- Storage waste

- Too many unused containers
- System clutter

Example in CI/CD:

Run container → Execute tests → Remove container

---

3. Important `docker rm` Options (Sub Commands)

1️⃣ Remove Multiple Containers

You can remove multiple containers at once.

docker rm container1 container2 container3

Example:

docker rm web_app db_container redis_container

Use case:

When cleaning multiple stopped containers.

2️⃣ Force Remove Container

Command:

docker rm -f CONTAINER_NAME

or

docker rm --force CONTAINER_NAME

This **forces removal even if container is running**.

Example:

docker rm -f my_app

What happens internally:

1. Docker **stops the container**
2. Then **removes it**

Use case:

- When container **refuses to stop**
- CI/CD cleanup scripts

Interview Tip:

`docker rm -f` **= docker stop + docker rm**

3️⃣ Remove Container with Volumes

Command:

docker rm -v CONTAINER_NAME

Example:

docker rm -v mysql_container

What it does:

- Removes container
- Also removes **anonymous volumes attached to container**

Important:

Named volumes are **not removed**.

Use case:

Prevent unused volumes from filling disk.

4️⃣ Remove Containers Using IDs

Example:

docker rm 4f8c2a91d3c2

Container IDs are obtained from:

docker ps -a

---

4. Removing All Stopped Containers

Command:

docker container prune

This removes **all stopped containers**.

Example output:

Deleted Containers:
abc123
def456

Use case:

System cleanup.

Common DevOps maintenance command.

---

5. Remove All Containers (Advanced)

Command:

docker rm $(docker ps -aq)

Explanation:

docker ps -aq

Lists **all container IDs**

$( )

Shell substitution.

Full workflow:

1. Get all container IDs
2. Pass them to `docker rm`

Example:

docker rm -f $(docker ps -aq)

Removes **all containers including running ones**.

---

6. Container Lifecycle (Important for Interviews)

The lifecycle explains **when `docker rm` is used**.

```
docker create
    ↓
docker start
    ↓
docker stop
    ↓
docker rm
```

Example workflow:

```
docker run -d nginx
docker stop container_id
```

docker rm container_id

---

7. Difference Between Similar Commands

Command	Purpose
`docker stop`	Stops running container
`docker kill`	Immediately stops container
`docker rm`	Removes stopped container
`docker container prune`	Removes all stopped containers
`docker rmi`	Removes images

Interview question often asked:

Difference between `docker rm` and `docker rmi`

Answer:

- `docker rm` → removes **containers**
- `docker rmi` → removes **images**

---

8. Best Beginner Workflow Example

Run nginx container:

docker run -d --name mynginx nginx

Check containers:

docker ps

Stop container:

docker stop mynginx

Remove container:

docker rm mynginx

---

9. Real DevOps + SDET Scenario

Imagine **automation tests running inside containers**.

Pipeline steps:

1. Pull test image
2. Run container
3. Execute tests
4. Collect reports
5. Remove container

Command used:

docker rm -f test_container

Why?

To ensure **clean environment for next test run**.

---

10. Most Asked Docker Interview Questions (Related)

Q1: Can you remove a running container?

Answer:

Yes using:

docker rm -f container_name

Q2: What happens if you try removing running container?

Error occurs.

Example:

Error response from daemon: You cannot remove a running container

---

Q3: Does `docker rm` delete images?

No.

It only deletes **containers**.

Q4: How do you remove all stopped containers?

Command:

docker container prune

---

11. Pro Tip (Important for DevOps)

To see **all containers including stopped**:

docker ps -a

Always check before deleting.

---

12. Visual Understanding

Think like this:

Docker Image
↓
Container (Running)
↓
Container (Stopped)
↓
docker rm

↓
Container Deleted

Image still remains.

✅ **Key takeaway**

`docker rm` is used to:

- Remove stopped containers
- Clean environment
- Manage container lifecycle
- Free resources

docker images

1. What `docker images` Command Does

`docker images` shows **all Docker images stored locally on your system**.

A **Docker image** is a **read-only template used to create containers**.

Example command:

docker images

Example Output:

```
REPOSITORY      TAG       IMAGE ID       CREATED        SIZE
nginx           latest    605c77e624dd   2 weeks ago    142MB
node            18        2e3c1a9a4c7f   3 weeks ago    1.1GB
ubuntu          22.04     a8780b506fa4   4 weeks ago    77MB
```

Column Meaning

Column	Meaning
REPOSITORY	Image name

TAG	Version of image
IMAGE ID	Unique identifier
CREATED	When image was created
SIZE	Image size

💡 **Key Concept**

Docker Image → Template
Docker Container → Running instance of image

---

2. Most Common Alternative Command

Modern Docker CLI also supports:

docker image ls

This is **the newer structured command**.

Command	Meaning
`docker images`	Old but still widely used
`docker image ls`	Official modern command

They produce the **same output**.

---

3. Important Flags (Sub Options) of `docker images`

3.1 Show All Images (including intermediate)

docker images -a

Example:

docker images -a

What it does

Shows **all images including intermediate layers created during build**.

Use Case

Useful when:

- Debugging **Docker build issues**
- Checking **build cache layers**

Example scenario:

Dockerfile build creates multiple layers

You can see them using `-a`.

3.2 Show Only Image IDs

docker images -q

Example:

docker images -q

Output:

605c77e624dd
2e3c1a9a4c7f
a8780b506fa4

Use Case

Useful when performing **bulk operations**.

Example: Remove all images

```
docker rmi $(docker images -q)
```

Explanation:

docker images -q → returns only image IDs
docker rmi → removes images

3.3 Filter Images

docker images --filter

Example:

```
docker images --filter "dangling=true"
```

What are dangling images?

Images with:

<none> repository
<none> tag

These are **unused images left after builds**.

Example output:

```
REPOSITORY   TAG     IMAGE ID       CREATED        SIZE
<none>       <none>  a34f1d34c9c1   2 days ago     210MB
```

Use Case

Cleaning unused images.

Example:

```
docker image prune
```

3.4 Format Output

docker images --format

Example:

```
docker images --format "{{.Repository}}:{{.Tag}}"
```

Output:

```
nginx:latest
node:18
ubuntu:22.04
```

Use Case

Automation scripts.

Example:

DevOps pipelines.

3.5 Show Digests

docker images --digests

Example:

docker images --digests

Output:

```
REPOSITORY  TAG     DIGEST
nginx       latest  sha256:abc123...
```

What is Digest?

A **secure hash identifying an image version**.

Used in **secure deployments**.

Example:

docker pull nginx@sha256:abc123

---

4. Related Commands (Important for Interviews)

These commands work together with `docker images`.

4.1 Pull an Image

docker pull nginx

What happens:

Docker Hub → Download image → Store locally

Now check:

docker images

Output:

nginx latest

4.2 Remove Image

docker rmi nginx

Example:

docker rmi nginx:latest

Remove by ID:

docker rmi 605c77e624dd

4.3 Inspect Image

docker image inspect nginx

Shows:

- Metadata

- Layers
- Architecture
- Environment variables

Useful for **debugging**.

4.4 Show Image History

docker history nginx

Example output:

```
IMAGE        CREATED       CREATED BY
605c77e624dd 2 weeks ago   /bin/sh -c apt-get update
```

Use Case

Understanding:

Dockerfile build layers

Very important in interviews.

4.5 Remove Dangling Images

docker image prune

Deletes unused images.

Example:

docker image prune -a

Removes **all unused images**.

---

5. Real Workflow Example

Example DevOps workflow.

Step 1: Pull Image

docker pull node:18

Step 2: Verify Image

docker images

Output:

node 18 2e3c1a9a4c7f

Step 3: Run Container

docker run -d -p 3000:3000 node:18

Step 4: Check Images

docker images

Step 5: Remove Unused Images

docker image prune

---

6. Important Concepts Beginners Must Know

Image Naming Convention

repository:tag

Example:

nginx:latest
node:18

mysql:8.0

Default Tag

If tag not specified:

docker pull nginx

Docker assumes:

nginx:latest

Image Layers

Docker images are **layered**.

Example:

Base Image (Ubuntu)
↓
Install packages
↓
Copy code
↓
Run command

Each step = **layer**

Benefits:

- Faster builds
- Caching
- Smaller updates

---

7. Real DevOps Use Cases

Use Case 1 — CI/CD pipelines

Check if required image exists.

docker images

Use Case 2 — Cleaning disk

Docker images consume space.

docker image prune

Use Case 3 — Debugging builds

Check intermediate layers.

docker images -a

Use Case 4 — Automation scripts

Get image IDs.

docker images -q

---

8. Interview Questions (Very Important)

Q1: What does `docker images` do?

Answer

`docker images` lists all Docker images stored locally along with repository name, tag, image ID, creation time, and size.

Q2: Difference between `docker images` and `docker ps`

Command	Shows

docker images	Available images
docker ps	Running containers

Q3: Difference between image and container

Image	Container
Template	Running instance
Read-only	Writable
Used to create containers	Executes application

Example:

docker run nginx

Creates container from **nginx image**.

Q4: What are dangling images?

Images with:

<none>:<none>

Created during builds and not tagged.

Remove them using:

docker image prune

Q5: Why do Docker images have layers?

Benefits:

- Faster builds

- Efficient storage
- Reusability
- Caching

---

9. Pro Tips to Crack Interviews

Memorize these **5 commands**.

```
docker images
docker image ls
docker image inspect
docker history
docker image prune
```

Also remember:

```
docker images -q
docker images -a
docker images --filter
```

These are **frequently asked in DevOps interviews**.

---

10. Best Visualization (Beginner Friendly)

```
Docker Hub
   ↓
docker pull nginx
   ↓
Local Images
   ↓
docker images
   ↓
docker run nginx
   ↓
Container Created
```

docker run nginx

Below is a **beginner-friendly but interview-ready explanation** of the command:

```
docker run nginx
```

I'll explain it like a **Docker mentor guiding a beginner**, but also include **interview points recruiters expect**.

1️⃣ What is Docker (1-line interview answer)

Docker is a containerization platform that packages an application and its dependencies into a **lightweight container** so it runs **consistently across environments** (dev, test, production).

2️⃣ What happens when you run

docker run nginx

Docker performs **5 automatic steps**:

1️⃣ Checks if the **nginx image** exists locally
2️⃣ If not → **pulls image from Docker Hub**
3️⃣ Creates a **container from the image**
4️⃣ Starts the container
5️⃣ Runs the **default nginx process**

So internally this command equals:

docker pull nginx
docker create nginx
docker start <container-id>

💡 **Interview Tip**

`docker run` = **pull + create + start**

3️⃣ Prerequisites before running `docker run nginx`

Before running the command you must have:

1️⃣ Docker installed

Install Docker Engine.

Check:

docker --version

Example output:

Docker version 25.x.x

2️⃣ Docker daemon running

Docker works with **Client → Daemon architecture**

Check:

docker info

If daemon not running:

Linux

sudo systemctl start docker

Windows/Mac → start Docker Desktop

3️⃣ Internet access

If the image is **not locally available**, Docker pulls it from:

👉 **Docker Hub**

4️⃣ Permission to run docker

Linux users sometimes need:

sudo docker run nginx

or add user to docker group.

4️⃣ What is nginx?

NGINX

NGINX is a **high-performance web server and reverse proxy**.

Use cases:

- Web hosting
- Load balancing
- Reverse proxy
- Static file serving

Docker image contains:

Linux OS
+
nginx installed
+
default configuration

5️⃣ What happens after running the command

Run:

docker run nginx

Output example:

```
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
...
Status: Downloaded newer image for nginx:latest
```

Container runs but **you can't access it yet** because port is not exposed.

---

6 Most common beginner command (important)

Expose container port

docker run -p 8080:80 nginx

Explanation:

```
-p 8080:80
|  |   |
|  |   → container port
|  → host port
→ publish port
```

Meaning:

localhost:8080 → nginx container:80

Now open browser:

http://localhost:8080

You will see nginx welcome page.

---

7 Run container in background

docker run -d -p 8080:80 nginx

Explanation:

-d → detached mode (background)

Without `-d`, terminal gets blocked.

---

8 Name your container

Instead of random names:

docker run -d -p 8080:80 --name my-nginx nginx

Benefits:

docker stop my-nginx
docker logs my-nginx

Easier to manage.

---

9 Important `docker run` options (INTERVIEW FAVORITES)

Interactive terminal

docker run -it ubuntu bash

Explanation:

-i → interactive
-t → terminal

Used for debugging containers.

Environment variables

docker run -e APP_ENV=prod nginx

Used in production deployments.

Mount volumes

docker run -v /host/path:/container/path nginx

Example:

docker run -d -p 8080:80 -v $(pwd):/usr/share/nginx/html nginx

Use case:

Serve **custom HTML files**.

Restart policy

docker run -d --restart=always nginx

Container automatically restarts if it crashes.

🔟 Most important docker commands (beginner → interview)

List running containers

docker ps

Output:

CONTAINER ID IMAGE PORTS

List all containers

docker ps -a

Stop container

docker stop my-nginx

Start container

docker start my-nginx

Remove container

docker rm my-nginx

View logs

docker logs my-nginx

Very important for debugging.

Enter container shell

docker exec -it my-nginx bash

Used for troubleshooting.

---

11 Docker run command structure

General format:

docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Example:

docker run -d -p 8080:80 --name web nginx

Breakdown:

docker run → create & start container
-d → detached
-p → port mapping
--name → container name
nginx → image

1️⃣2️⃣ Real world use cases

1️⃣ Run web server quickly

docker run -d -p 80:80 nginx

Use case:

Temporary test server.

2️⃣ Serve static website

```
docker run -d -p 8080:80 \
-v $(pwd)/website:/usr/share/nginx/html \
nginx
```

3️⃣ Microservices deployment

Each service runs as **separate container**.

Example:

```
frontend container
backend container
database container
```

1️⃣3️⃣ Common mistakes beginners make

❌ Container exits immediately

Because **main process stops**.

Check logs:

docker logs container_id

❌ Port not accessible

Forgot `-p` mapping.

❌ Can't stop container

Use container name or ID.

1️⃣4️⃣ Key Docker concepts (interview must know)

Image

Blueprint for containers.

Example:

nginx
ubuntu
node

Container

Running instance of image.

Example:

nginx container running on port 8080

Docker Hub

Public image registry.

Example:

docker pull nginx

Volumes

Persistent storage.

Networks

Containers communicate using Docker networks.

---

15 Best interview answer (short version)

If interviewer asks:

What happens when you run `docker run nginx`?

Answer:

> When we run `docker run nginx`, Docker checks if the nginx image exists locally. If not, it pulls it from Docker Hub, then creates a container from the image and starts it using nginx as the default process.

---

16 Advanced knowledge (impress interviewers)

Run container with resource limits

docker run -d --memory="512m" --cpus="1.0" nginx

Limits CPU and RAM.

Run specific image version

docker run nginx:1.25

Never rely on `latest` in production.

1️⃣7️⃣ Golden Docker rule

Image = blueprint
Container = running instance

⭐ Pro tip for DevOps / SDET interviews

You should also know:

docker build
docker-compose
Dockerfile
container networking
CI/CD integration

docker pull nginx

Below is a **beginner → interview-ready explanation** of the command:

docker pull nginx

I'll explain it like a **Docker mentor would teach a beginner**, while also including **interview-relevant points**.

---

1. What is `docker pull nginx`?

Definition

`docker pull nginx` downloads the **Nginx image** from a Docker registry (by default **Docker Hub**) to your local machine.

Entity references:

- Docker
- Nginx
- Docker Hub

Simple meaning

> It downloads the ready-made container image of Nginx so you can run it as a container.

Example

docker pull nginx

Output will look like:

Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:xxxxxx
Status: Downloaded newer image for nginx:latest

---

2. Pre-requisites before running `docker pull`

Before running the command, these things must be ready:

1️⃣ Install Docker

Verify installation:

docker --version

Example output

Docker version 26.1.0

2️⃣ Docker daemon must be running

Check:

docker info

If Docker daemon is not running, start it:

Linux:

sudo systemctl start docker

Windows / Mac:

Start Docker Desktop.

3️⃣ Internet connection

Because the image will be downloaded from **Docker Hub registry**.

4️⃣ Optional: Docker login (for private images)

docker login

Needed when pulling from **private repositories**.

---

3. What happens internally when `docker pull nginx` runs?

Step-by-step workflow:

1️⃣ Docker CLI receives command

docker pull nginx

2️⃣ CLI communicates with **Docker daemon**

3️⃣ Docker daemon contacts **Docker Hub registry**

4️⃣ It searches for image

nginx:latest

5️⃣ Image layers are downloaded

6️⃣ Layers are stored locally

7️⃣ Image becomes available for container creation

Check downloaded images:

docker images

Example output

```
REPOSITORY   TAG       IMAGE ID       SIZE
nginx        latest    2c5e00f2a3     187MB
```

---

4. Understanding the command structure

General syntax:

docker pull [OPTIONS] NAME[:TAG|@DIGEST]

Explanation:

Part	Meaning
docker	Docker CLI
pull	download image
NAME	image name
TAG	version

Example

docker pull nginx:1.25

Pulls a **specific version**.

---

5. Important `docker pull` options (sub-commands)

1️⃣ Pull a specific version

docker pull nginx:1.25

Why?

Production systems should **never use latest tag**.

2️⃣ Pull from specific registry

Example:

docker pull docker.io/library/nginx

Explanation:

registry/repository/image

3️⃣ Pull image by digest

Most secure way.

docker pull nginx@sha256:abc123

Use case:

Ensures **exact image version**.

---

6. Alternative commands related to `docker pull`

1️⃣ `docker run` (automatically pulls image)

Example:

docker run nginx

If image not available locally → Docker automatically performs:

docker pull nginx

Then container starts.

2️⃣ `docker build`

Instead of pulling image, you can build your own.

Example Dockerfile:

```
FROM nginx
COPY index.html /usr/share/nginx/html
```

Build image:

```
docker build -t my-nginx .
```

3️⃣ `docker image pull`

Newer command format:

```
docker image pull nginx
```

Equivalent to:

```
docker pull nginx
```

---

7. Real-world use cases

1️⃣ Running web servers quickly

Example:

```
docker run -d -p 8080:80 nginx
```

Access:

http://localhost:8080

2️⃣ Microservices development

Developers pull images like:

```
nginx
mysql
redis
node
```

Then connect containers.

3️⃣ CI/CD pipelines

Used in pipelines like:

- Jenkins
- GitHub Actions
- GitLab

Example pipeline step:

```
docker pull nginx:1.25
```

Ensures correct environment.

4️⃣ Kubernetes deployments

Containers used in:

- Kubernetes

Example YAML:

```
containers:
- name: nginx
 image: nginx:1.25
```

Kubernetes automatically pulls the image.

---

8. Commands used after `docker pull`

1️⃣ Verify image

docker images

2️⃣ Run container

docker run nginx

3️⃣ Run container in background

docker run -d nginx

4️⃣ Map ports

docker run -d -p 8080:80 nginx

5️⃣ See running containers

docker ps

6️⃣ Stop container

docker stop <container_id>

7️⃣ Remove container

docker rm <container_id>

---

9. Layer concept (Important Interview Question)

Docker images are **layered**.

Example:

nginx image layers

Nginx binaries
Linux libraries
Base OS (Alpine / Debian)

When pulling:

- Only **missing layers download**
- Existing layers are reused

Benefits:

✔ Faster downloads
✔ Less storage
✔ Efficient caching

---

10. Difference: Image vs Container (Common Interview Question)

Image	Container
Template	Running instance
Read-only	Read-write
Built or pulled	Created from image

Example:

docker pull nginx

downloads **image**

docker run nginx

creates **container**

---

11. Best practices (Important for interviews)

1️⃣ Avoid latest tag

Bad:

docker pull nginx

Good:

docker pull nginx:1.25

2️⃣ Use lightweight images

Example:

nginx:alpine

Smaller size.

3️⃣ Remove unused images

docker image prune

4️⃣ Check image vulnerability

Tools:

- Trivy
- Clair

---

12. Example: Complete beginner workflow

Step 1 — Pull image

docker pull nginx

Step 2 — Verify

docker images

Step 3 — Run container

docker run -d -p 8080:80 nginx

Step 4 — Check running container

docker ps

Step 5 — Access in browser

http://localhost:8080

You will see **Nginx welcome page**.

---

13. Interview-ready explanation (30-second answer)

> "`docker pull nginx` downloads the nginx container image from Docker Hub to the local Docker host. Docker checks the registry, downloads missing layers, and stores them locally. Once the image is available, we can create containers using `docker run`. It's commonly used in development, CI/CD pipelines, and container orchestration systems like Kubernetes."

---

14. Pro tips that impress interviewers

Mention these:

✔ Image layers
✔ Tags and versions
✔ Registries (Docker Hub / private registry)
✔ Digest based pulling

✔ Security scanning
✔ Pull vs Run behavior

docker run ubuntu

Command; docker run ubuntu
If you want to run a docker container from ubuntu image, it run the instance of ubuntu image, and exists immediately. If you list the running containers using
Command: docker ps
Its empty
Command to check all the containers, including those are stopped,
Command: docker ps -a
Ubuntu is in exited state.

```
kiran@LAPTOP-BON7O57K:~$ docker run ubuntu
Unable to find image 'ubuntu:latest' locally
^Ckiran@LAPTOP-BON7O57K:~$ docker run ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
01d7766a2e4a: Pull complete
fd8cda969ed2: Download complete
Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495
eff4f9
Status: Downloaded newer image for ubuntu:latest
kiran@LAPTOP-BON7O57K:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
kiran@LAPTOP-BON7O57K:~$ docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED              STATUS
                      PORTS     NAMES
d27ee5b8aa41   ubuntu        "/bin/bash"   About a minute ago   Exited (0
) About a minute ago             jovial_meninsky
b315f213d5f9   hello-world   "/hello"      16 hours ago         Exited (0
) 16 hours ago                   great_nash
kiran@LAPTOP-BON7O57K:~$
```

Containers are not meant to host the operating system. Containers are meant to run the specific task or process, such as to host the instance of web server, application server, or database, or for computation or analysis task, once the task is complete, container exits, container exists only if the task or process is alive in it.
If the webserver inside the container is stoped, or crashed then container exists,
If you want to see the content of the file inside a particular a particular container, to see the content in the etc/hosts file:

```
kiran@LAPTOP-BON7O57K:~$ docker run ubuntu sleep 500
^C
```

Use (ctrl + c) multiple times.

```
got 3 SIGTERM/SIGINTs, forcefully exiting
kiran@LAPTOP-BON7O57K:~$ ^C
kiran@LAPTOP-BON7O57K:~$ ^C
kiran@LAPTOP-BON7O57K:~$ docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED          STATUS                     PORTS     NAMES
7bfbf30cf2e4   ubuntu        "sleep 500"   28 seconds ago   Up 28 seconds                        keen_turing
06a0b42034b8   ubuntu        "sleep 5"     3 minutes ago    Exited (0) 3 minutes ago             wonderful_lehmann
```

```
kiran@LAPTOP-BON7O57K:~$ docker exec keen_turing cat /etc/hosts
127.0.0.1       localhost
::1     localhost ip6-localhost ip6-loopback
fe00::  ip6-localnet
ff00::  ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
172.17.0.2      7bfbf30cf2e4
kiran@LAPTOP-BON7O57K:~$
```

Command: docker run kodekloud/simple-webapp

Which is in attached mode, it run and it stops when **ctrl + c** is clicked. In between you can't give command.

Command in detached mode:

```
^Ckiran@LAPTOP-BON7O57K:~$ docker run -d kodekloud/simple-webapp
d4e748c16cfd9f9a389fe2f23a41497fcf4fc35493c7b39bafe71c478b3545c4
kiran@LAPTOP-BON7O57K:~$
```

If you want to attach to the webapp, which is in detached mode: no need to provide entire ID

```
^Ckiran@LAPTOP-BON7O57K:~$ docker run -d kodekloud/simple-webapp
d4e748c16cfd9f9a389fe2f23a41497fcf4fc35493c7b39bafe71c478b3545c4
kiran@LAPTOP-BON7O57K:~$ docker attach d4e748c16cfd
```

```
kiran@LAPTOP-BON7O57K:~$ docker run kodekloud/simple-webapp
Unable to find image 'kodekloud/simple-webapp:latest' locall
latest: Pulling from kodekloud/simple-webapp
e337be014a61: Pull complete
f0d690b9e495: Pull complete
fac5d45ad062: Pull complete
4fe2ade4980c: Pull complete
7454877e71d0: Pull complete
7cf6a1d62200: Pull complete
dd9b067ef6fd: Pull complete
Digest: sha256:e5355b4c7804f453d79de75d6659ee702eeebbe30c02c
Status: Downloaded newer image for kodekloud/simple-webapp:l
 This is a sample web application that displays a colored ba
 A color can be specified in two ways.

 1. As a command line argument with --color as the argument.
 2. As an Environment variable APP_COLOR. Accepts one of red
 3. If none of the above then a random color is picked from
 Note: Command line argument precedes over environment varia


No command line argument or environment variable. Picking a
 * Serving Flask app "app" (lazy loading)
 * Environment: production
   WARNING: Do not use the development server in a productio
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

Use link to test yourself:
https://learn.kodekloud.com/courses/freecodecamp-labs-docker-for-the-absolute-beginner?utm_source=youtube&utm_medium=labs&utm_campaign=docker

To run the redis:

```
kiran@LAPTOP-BON7O57K:~$ docker run redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
141e1fc22067: Pull complete
214ecf6e5c87: Pull complete
ae77945b0371: Pull complete
89b1b12d7109: Pull complete
f25f5c85b90d: Pull complete
4f4fb700ef54: Pull complete
206356c42440: Pull complete
e798a26a6510: Download complete
4a4db1a35d5b: Download complete
Digest: sha256:1c054d54ecd1597bba52f4304bca5afbc5565ebe614c5b3d7dc5b7f8a0cd
Status: Downloaded newer image for redis:latest
Starting Redis Server
1:C 13 Mar 2026 09:42:34.948 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0O
1:C 13 Mar 2026 09:42:34.948 * Redis version=8.6.1, bits=64, commit=0000000
1:C 13 Mar 2026 09:42:34.948 * Configuration loaded
1:M 13 Mar 2026 09:42:34.948 * monotonic clock: POSIX clock_gettime
1:M 13 Mar 2026 09:42:34.948 * Running mode=standalone, port=6379.
1:M 13 Mar 2026 09:42:34.949 * <bf> RedisBloom version 8.6.0 (Git=unknown)
```

If you want to run the redis with specific version other than default tag as latest: **use tags**

```
kiran@LAPTOP-BON7O57K:~$ docker run redis:4.0
Unable to find image 'redis:4.0' locally
4.0: Pulling from library/redis
40e134f79af1: Pull complete
54fec2fa59d0: Pull complete
9c94e11103d9: Pull complete
8ce1bab2086c: Pull complete
7988789e1fb7: Pull complete
04ab1bfc453f: Pull complete
Digest: sha256:2e03fdd159f4a08d2165ca1c92adde438ae4e3e6b
Status: Downloaded newer image for redis:4.0
1:C 13 Mar 09:46:23.326 # oO0OoO0OoO0Oo Redis is starti
1:C 13 Mar 09:46:23.326 # Redis version=4.0.14, bits=64,
1:C 13 Mar 09:46:23.326 # Warning: no config file speci
```

Visit: dockerhub.com or https://hub.docker.com/

Search for redis, go to any image and go to overview:

Supported tags and respective `Dockerfile` links

- `8.6.1`, `8.6`, `8`, `8.6.1-trixie`, `8.6-trixie`, `8-trixie`, `latest`, `trixie`
- `8.6.1-alpine`, `8.6-alpine`, `8-alpine`, `8.6.1-alpine3.23`, `8.6-alpine3.23`, `8-alpine3.23`, `al`
  `alpine3.23`
- `8.4.2`, `8.4`, `8.4.2-trixie`, `8.4-trixie`
- `8.4.2-alpine`, `8.4-alpine`, `8.4.2-alpine3.22`, `8.4-alpine3.22`
- `8.2.5`, `8.2`, `8.2.5-bookworm`, `8.2-bookworm`
- `8.2.5-alpine`, `8.2-alpine`, `8.2.5-alpine3.22`, `8.2-alpine3.22`
- `8.0.6`, `8.0`, `8.0.6-bookworm`, `8.0-bookworm`
- `8.0.6-alpine`, `8.0-alpine`, `8.0.6-alpine3.21`, `8.0-alpine3.21`
- `7.4.8`, `7.4`, `7`, `7.4.8-bookworm`, `7.4-bookworm`, `7-bookworm`
- `7.4.8-alpine`, `7.4-alpine`, `7-alpine`, `7.4.8-alpine3.21`, `7.4-alpine3.21`, `7-alpine3.21`

Which has many version groups.

For shell scripting:

```
kiran@LAPTOP-BON7O57K:~$ nano ask_name.sh
```

Write the script:

```
kiran@LAPTOP-BON7O57K: ~

  GNU nano 7.2
#!/bin/bash

echo "Enter your name:"
read name

echo "Hello $name, welcome to Shell Scripting!"

^G Help        ^O Write Out   ^W Where Is    ^K Cu
^X Exit        ^R Read File   ^\ Replace     ^U Pa
```

Ctrl+x -> **y** -> **Enter**
To make it executable: chmod +x ask_name.sh
To run the script: ./ask_name.sh

```
kiran@LAPTOP-BON7O57K:~$ chmod +x ask_name.sh
kiran@LAPTOP-BON7O57K:~$ ./ask_name.sh
Enter your name:
Routh kiran Babu
Hello Routh kiran Babu, welcome to Shell Scripting!
```

To dockerise this application,
It doesn't run in a interactive way:

```
kiran@LAPTOP-BON7O57K:~$ docker run kodekloud/simple-prompt-docker
Unable to find image 'kodekloud/simple-prompt-docker:latest' locally
latest: Pulling from kodekloud/simple-prompt-docker
ed595ae334dc: Pull complete
Digest: sha256:8da0602f5beaa472d40efd7d56754006f7a4ce1d57fff80f99ac0f5a91236d7d
Status: Downloaded newer image for kodekloud/simple-prompt-docker:latest
Welcome! Please enter your name:
Hello and Welcome !
```

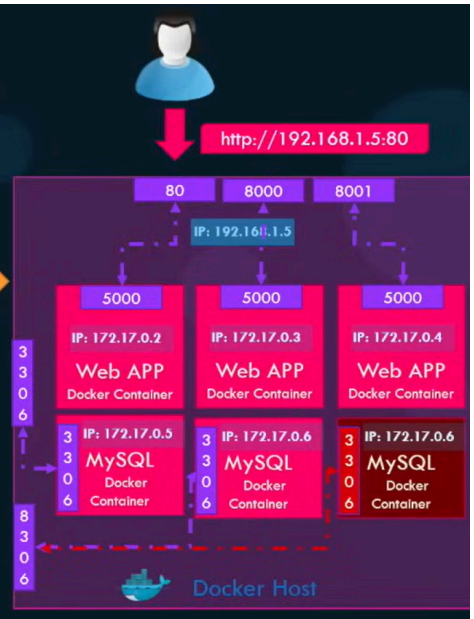
To take input from user:

```
kiran@LAPTOP-BON7O57K:~$ docker run -i kodekloud/simple-prompt-docker
Welcome! Please enter your name: routh kiran babu

Hello and Welcome routh kiran babu!
kiran@LAPTOP-BON7O57K:~$ docker run -it kodekloud/simple-prompt-docker
Welcome! Please enter your name: Routh Kiran Babu

Hello and Welcome Routh Kiran Babu!
kiran@LAPTOP-BON7O57K:~$
```

Run – PORT mapping

```
docker run kodekloud/webapp
 * Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
```

http://172.17.0.2:5000

Internal IP

```
docker run -p 80:5000 kodekloud/simple-webapp
docker run -p 8000:5000 kodekloud/simple-webapp
docker run -p 8001:5000 kodekloud/simple-webapp
docker run -p 3306:3306 mysql
docker run -p 8306:3306 mysql
docker run -p 8306:3306 mysql
```